OS_project3

12912 2022090000

정인호

1. Design (설계)

본 과제의 핵심 목표는 xv6 운영체제에 Memory-Mapped Files(mmap, munmap) 기능을 추가하여, 프로세스가 파일 시스템의 데이터를 메모리 배열처럼 다룰 수 있게 하는 것이다. 전체 시스템은 메모리 낭비를 막기 위해 **지연 할당(Lazy Allocation)** 방식으로 설계하였다.

1. **VMA (Virtual Memory Area) 장부 관리:** mmap 시스템 콜이 호출되면 즉시 메모리와 디스크 I/O 를 수행하지 않고, 프로세스 구조체(struct proc)의 vmas 배열에 매핑할 가상 주소, 길이, 파일, 권한 등의 메타데이터만 기록합니다.
2. **가상 주소 할당 정책 (Top-Down):** sbrk 에 의해 위로 자라나는 힙(Heap, p->sz) 영역과의 충돌을 방지하기 위해, mmap 영역은 사용자 주소 공간의 최상단인 TRAPFRAME 바로 아래부터 시작하여 아래 방향(Downward)으로 할당되도록 설계했습니다.
3. **Lazy Page Fault (지연 로딩):** 유저 프로그램이 매핑된 주소에 최초로 접근하여 Page Fault 가 발생하면, mmapfault 핸들러가 이를 가로채어 물리 페이지를 할당하고 파일에서 데이터를 읽어와 페이지 테이블에 연결합니다.
4. **동기화 및 자원 해제:** munmap 또는 프로세스 종료(kexit) 시, MAP_SHARED 플래그가 설정된 변경된 페이지는 파일(Disk)에 Write-back 하여 데이터 일관성을 유지하고 커널 자원(물리 페이지, VMA 슬롯, 파일 참조)을 안전하게 반환합니다.

2. Implementation (구현 상세)

2.1. sys_mmap (kernel/sysfile.c)

```
uint64
```

```
sys_mmap(void)
{
    int length, prot, flags, fd, offset;
    struct file *f;
    struct proc *p = myproc();


    argint(0, &length);
    argint(1, &prot);
    argint(2, &flags);
    if(argfd(3, &fd, &f) < 0) {
        return -1;
    }
    argint(4, &offset);


    if(length <= 0 || length % PGSIZE != 0) return -1;
    if(offset < 0 || offset % PGSIZE != 0) return -1;
    if((prot & ~(PROT_READ | PROT_WRITE)) != 0 || prot == 0) return -1; // PROT_NONE  불가
    if(flags != MAP_SHARED && flags != MAP_PRIVATE) return -1;


    if(!f->readable && (prot & PROT_READ)) return -1;
    if((prot & PROT_WRITE) && (flags & MAP_SHARED) && !f->writable) return -1;


    struct vma *v = 0;
    for(int i = 0; i < NVMA; i++){
        if(p->vmas[i].used == 0){
            v = &p->vmas[i];
            break;
        }
    }
    if(v == 0) return -1;


    uint64 maplen = PGROUNDUP(length);
    uint64 alloc_addr = TRAPFRAME;

    for(int i = 0; i < NVMA; i++){
        if(p->vmas[i].used){
```

```
            if(p->vmas[i].addr < alloc_addr){
                alloc_addr = p->vmas[i].addr;
            }
        }
    }

    alloc_addr -= maplen;


    if(alloc_addr < p->sz) return -1;


    v->used = 1;
    v->addr = alloc_addr;
    v->length = maplen;
    v->prot = prot;
    v->flags = flags;
    v->offset = offset;

    v->f = f;
    filedup(v->f);

    return v->addr;
}
```

유저가 요청한 매핑 정보를 검증하고 VMA 슬롯을 할당한다.

- **인자 파싱 및 검증:** 과제 API 명세에 맞추어 5 개의 인자(length, prot, flags, fd, offset)를 파싱한다. length 와 offset 의 페이지 정렬 여부, 유효하지 않은 권한(PROT_NONE), 파일의 읽기/쓰기 권한과 MAP_SHARED 충돌 여부를 엄격하게 검증하여 잘못된 요청을 선제 차단했다.
- **주소 할당 및 힙 오버랩 방지:** 현재 사용 중인 VMA 영역 중 가장 낮은 주소를 찾아 그 아래로 가상 주소를 계산(alloc_addr)한다. 이때 계산된 주소가 힙 영역(p->sz)을 침범할 경우 에러(-1)를 반환하도록 방어 로직을 추가했다.

- **파일 참조 증가:** mmap 이 유지되는 동안 파일 구조체가 해제되지 않도록 filedup(f)를 호출해 레퍼런스 카운트를 늘려주었다.

2.2. mmapfault (kernel/sysfile.c)

```
int
mmapfault(uint64 va)
{
    struct proc *p = myproc();
    struct vma *v = 0;
    for(int i = 0; i < NVMA; i++){
        if(p->vmas[i].used && va >= p->vmas[i].addr && va < p->vmas[i].addr + p->vmas[i].length){
            v = &p->vmas[i];
            break;
        }
    }

    if(v == 0){
        return -1;
    }

    uint64 a = PGROUNDDOWN(va);
    if(walkaddr(p->pagetable, a) != 0) {
        return -1;
    }


    void *pa = kalloc();
    if(pa == 0) return -1;
    memset(pa, 0, PGSIZE);

    int offset = v->offset + (a - v->addr);

    ilock(v->f->ip);
    readi(v->f->ip, 0, (uint64)pa, offset, PGSIZE);
    iunlock(v->f->ip);
    int perm = PTE_U;
    if(v->prot & PROT_READ)    {
        perm |= PTE_R;
```

```
    }
    if(v->prot & PROT_WRITE) {
        perm |= PTE_R | PTE_W;
    }

    if(mappages(p->pagetable, a, PGSIZE, (uint64)pa, perm) != 0){
        kfree(pa);
        return -1;
    }

    return 0;
}
```

실제 메모리 접근 시 발생하는 트랩(Page Fault)을 처리한다.

- **중복 폴트 및 권한 위반 방지:** walkaddr 을 사용하여 해당 주소에 이미 물리 페이지가 매핑되어 있는지 선제 검사한다. 이미 매핑되어 있다면 Read-only 영역에 Write 를 시도하는 등의 권한 위반(Protection Fault)이므로 -1 을 반환해 mappages: remap 커널 패닉을 방지했다.
- **파일 로딩 및 RISC-V 권한 설정:** kalloc 으로 물리 메모리를 확보하고 readi 를 통해 디스크 오프셋으로부터 PGSIZE 만큼 데이터를 읽는다. 이후 RISC-V 아키텍처에서 쓰기 전용(W=1, R=0) 매핑이 불가능한 하드웨어적 한계를 고려하여, PROT_WRITE 요청 시 PTE_R | PTE_W 권한을 함께 부여하여 매핑했다.

2.3. sys_munmap (kernel/sysfile.c)

```
uint64
sys_munmap(void)
{
    uint64 addr;
    int length;
    struct proc *p = myproc();
    struct vma *v = 0;

    argaddr(0, &addr);
```

```
argint(1, &length);


if(length <= 0 || length % PGSIZE != 0 || addr % PGSIZE != 0) {
    return -1;
}


uint64 end = addr + length;
if(end < addr) {
    return -1;
}

for(int i = 0; i < NVMA; i++){
    if(p->vmas[i].used && addr >= p->vmas[i].addr && end <= p->vmas[i].addr + p->vmas[i].length){
        v = &p->vmas[i];
        break;
    }
}

if(v == 0) return -1;


if(addr != v->addr && end != v->addr + v->length){
    return -1;
}

if((v->flags & MAP_SHARED) && v->f->writable){
    for(uint64 a = addr; a < addr + length; a += PGSIZE){
        uint64 pa = walkaddr(p->pagetable, a);
        if(pa != 0){
            int file_offset = v->offset + (a - v->addr);

            begin_op();
            ilock(v->f->ip);
            writei(v->f->ip, 0, pa, file_offset, PGSIZE);
            iunlock(v->f->ip);
            end_op();
        }
    }
```

```
    }

    for(uint64 a = addr; a < addr + length; a += PGSIZE){
        if(walkaddr(p->pagetable, a) != 0){
            uvmunmap(p->pagetable, a, 1, 1);
        }
    }

    if(addr == v->addr && length == v->length){
        v->used = 0;
        fileclose(v->f);
        v->f = 0;
    } else if(addr == v->addr){
        v->addr += length;
        v->length -= length;
        v->offset += length;
    } else if(addr + length == v->addr + v->length){
        v->length -= length;
    } else {
        return -1;
    }

    return 0;
}
```

매핑된 메모리 영역을 해제하고 파일과 동기화한다.

- **오버플로우 및 Middle-hole 차단:** 정수 오버플로우 공격을 막기 위해 end < addr 검사를 추가했다. 또한 명세서의 요구사항에 따라 VMA 영역의 중간 부분만 도려내는 'Middle-hole' 해제 요청이 들어오면 Write-back 이 수행되기 전에 즉시 에러 처리했다.
- **Write-back 및 메모리 반환:** MAP_SHARED 권한이고 파일이 쓰기 가능할 경우, walkaddr 로 실제 물리 페이지가 할당된(사용된) 부분만 writei 를 통해 디스크에 기록한다. 이후 uvmunmap 을 호출해 페이지 테이블 연결을 끊고 메모리를 반환한다.

- **VMA 부분 해제:** 해제 요청이 영역의 앞/뒤 일부인 경우 addr, length, offset 을 갱신하고, 영역 전체가 해제된 경우에만 fileclose 를 호출해 파일 참조를 반환했다.

2.4. kfork & kexit (kernel/proc.c)

- **kfork:** 자식 프로세스 생성 시 부모의 vmas 배열을 동일하게 복사하고, 복사된 각 VMA 파일의 참조 횟수를 filedup 으로 증가시킨다. 물리 메모리는 복사하지 않아 자식도 지연 할당한다.

```
for(i = 0; i < NVMA; i++) {
        if(p->vmas[i].used) {
            np->vmas[i] = p->vmas[i];
            filedup(np->vmas[i].f);
        }
    }
```

- **kexit:** 프로세스가 종료될 때 유저가 munmap 을 호출하지 않고 끝나는 경우를 대비하여, 남아있는 모든 VMA 슬롯을 순회하며 MAP_SHARED 플래그인 경우 Write-back 을 수행한 뒤 모든 물리 메모리와 VMA 구조체를 강제로 해제하여 Memory Leak 을 방지했다.

```
void
kexit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
        panic("init exiting");

    for(int i = 0; i < NVMA; i++){
        if(p->vmas[i].used){
            struct vma *v = &p->vmas[i];

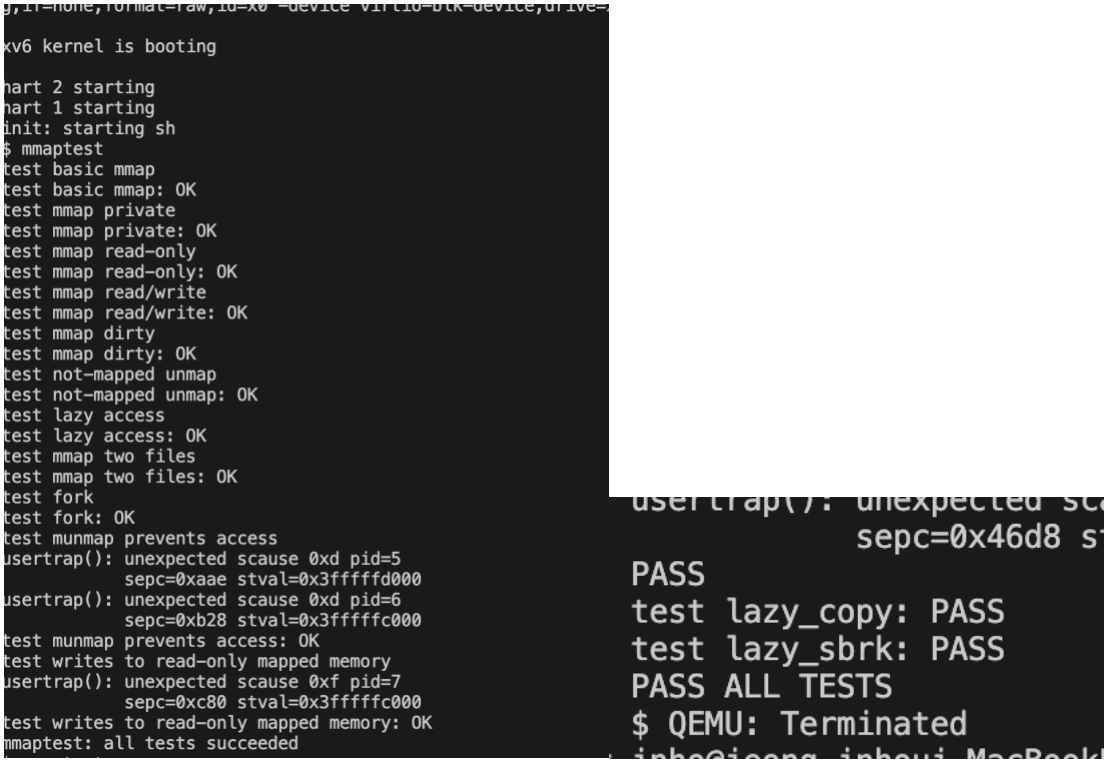
            if((v->flags & MAP_SHARED) && v->f->writable){
                for(uint64 a = v->addr; a < v->addr + v->length; a += PGSIZE){
                    uint64 pa = walkaddr(p->pagetable, a);
                    if(pa != 0){
```

```
                    int file_offset = v->offset + (a - v->addr);
                    begin_op();
                    ilock(v->f->ip);
                    writei(v->f->ip, 0, pa, file_offset, PGSIZE);
                    iunlock(v->f->ip);
                    end_op();
                }
            }
        }

        for(uint64 a = v->addr; a < v->addr + v->length; a += PGSIZE){
            if(walkaddr(p->pagetable, a) != 0){
                uvmunmap(p->pagetable, a, 1, 1);
            }
        }

        fileclose(v->f);
        v->used = 0;
    }
}
```

3. Results (실행 결과)



문제 없이 모든 테스트를 통과한 것을 알 수 있다.